BI3 Technologies · Follow publication

Enable GPU Acceleration in Kubernetes Using NVIDIA GPU Operator

4 min read · Mar 28, 2025

Ashwin Durairaj Follow

Listen Share

Introduction

As AI and machine learning applications grow in scale and complexity, the demand for powerful GPU resources is higher than ever. Kubernetes provides a scalable way to manage GPU workloads across your cluster efficiently

> ***Note:*** *You don't need to install NVIDIA drivers or CUDA manually — the operator handles it for you by deploying the necessary components across your GPU nodes.*

Step 1: Install Helm on the Master Node

Helm is a tool that helps you install and manage pods and other resources in Kubernetes. It uses something called “charts,” which are like pre-made setup files that make deployment easier and more consistent.

We’ll use Helm to install the GPU Operator and related components.

To install Helm, run the below commands:

```
$ curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/master/
$ chmod 700 get_helm.sh
$ ./get_helm.sh
```

Step 2: Install the NVIDIA GPU Operator

The NVIDIA GPU Operator simplifies GPU management in Kubernetes. It includes essential components like the Node Feature Discovery (NFD) and the NVIDIA Driver Manager. GPU Operator enables faster node provisioning by automatically installing all necessary software components for running GPU-accelerated applications.

i. Create the GPU Operator Namespace

Begin by creating a new namespace for the GPU operator:

```
$ kubectl create ns gpu-operator
```

ii. Configure Pod Security Admission (Optional)

If your cluster uses Pod Security Admission (PSA), label the namespace to set the security policy to privileged:

```
$ kubectl label --overwrite ns gpu-operator pod-security.kubernetes.io/enforce=
```

iii. Check for Node Feature Discovery (NFD)

By default, Node Feature Discovery (NFD) is automatically deployed in the master and worker by the Nvidia Operator. If NFD is already running in the cluster, then

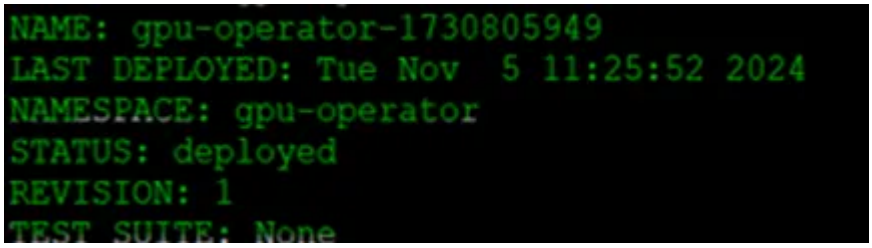
you must disable deploying NFD when you install the Operator

If NFD is already running on your cluster, you can skip deploying it again. To check if NFD is already deployed, use:

```
$ kubectl get nodes -o json | jq '.items[].metadata.labels | keys | any(startsw
```

iv. Add the NVIDIA Helm repository and install the GPU operator:

```
$ helm repo add nvidia https://helm.ngc.nvidia.com/nvidia && helm repo update
$ helm install --wait --generate-name \
  -n gpu-operator --create-namespace \
  nvidia/gpu-operator \
  --version=v25.3.0
```



Helm deployment status of the GPU Operator, confirming that it is deployed in the `gpu-operator` namespace

Step 3: Verify GPU Operator Deployment

To verify that the GPU operator was set up successfully, check the pods in the `**gpu-operator**` namespace:

```
$ kubectl get pods -n gpu-operator
```

```
root@k8master:/home/ubuntu# kubectl get pods -n gpu-operator
NAME                                                          READY   STATUS      RESTARTS   AGE
gpu-feature-discovery-6wtq5                                   1/1     Running     0          9m46s
gpu-operator-1731315579-node-feature-discovery-gc-bdcb6c5cgbc6w   1/1     Running     0          10m
gpu-operator-1731315579-node-feature-discovery-master-84967mttw   1/1     Running     0          10m
gpu-operator-1731315579-node-feature-discovery-worker-4hbk2   1/1     Running     0          10m
gpu-operator-1731315579-node-feature-discovery-worker-tq5nz   1/1     Running     0          10m
gpu-operator-79f94f69df-zrvrd                                 1/1     Running     0          10m
nvidia-container-toolkit-daemonset-7rfsf                      1/1     Running     0          9m47s
nvidia-cuda-validator-gj8g4                                   0/1     Completed   0          9m7s
nvidia-dcgm-exporter-zhfws                                    1/1     Running     0          9m47s
nvidia-device-plugin-daemonset-2p72t                          1/1     Running     0          9m47s
nvidia-operator-validator-glh6j                               1/1     Running     0          9m47s
```

Pods running inside ‘gpu-operator’ namespace

If all pods are running without errors, your GPU setup is complete!

Step 4: Test GPU Utilization with a Sample CUDA Application

To confirm that GPUs are correctly configured and can be utilized by containers, let’s deploy a sample CUDA application that performs a simple vector addition.

i. Create a `cuda-vectoradd.yaml` file

Save the following sample YAML configuration for a CUDA application:

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: cuda-vectoradd
spec:
  restartPolicy: OnFailure
  containers:
  - name: cuda-vectoradd
    image: "nvcr.io/nvidia/k8s/cuda-sample:vectoradd-cuda11.7.1-ubuntu20.04"
    resources:
      limits:
        nvidia.com/gpu: 1
```

ii. Deploy the Pod

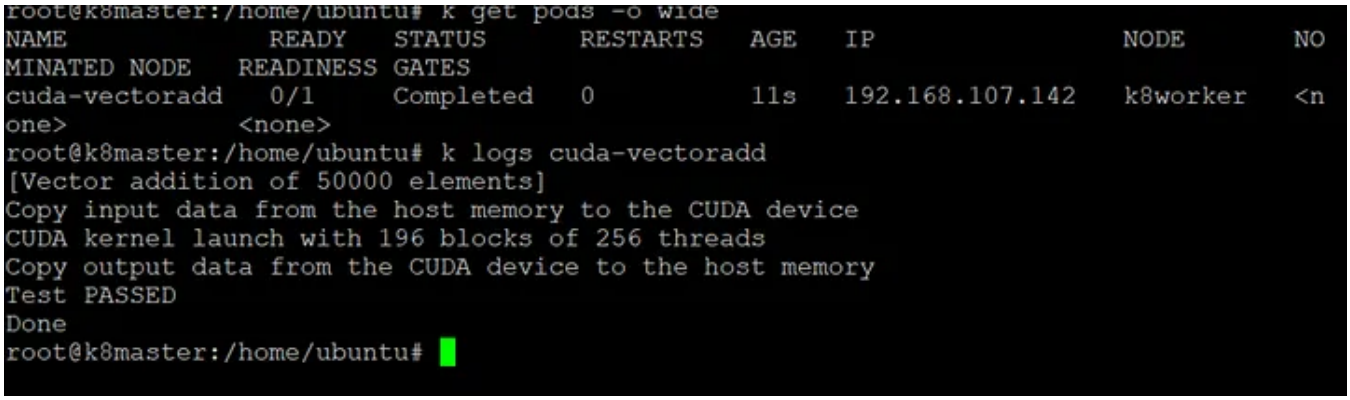
Run the following command to deploy the pod:

```
$ kubectl apply -f cuda-vectoradd.yaml
```

iii. View Logs to Confirm GPU Utilization

After the pod runs, check the logs to ensure it completed successfully:

```
$ kubectl logs cuda-vectoradd
```



Output from `kubectl logs` command for the `cuda-vectoradd` pod, showing the results of a sample GPU workload to verify CUDA functionality.

The output should indicate successful vector addition, verifying that the GPU works correctly.

Conclusion

With GPU support enabled through the NVIDIA GPU Operator, your Kubernetes cluster is now fully equipped to run GPU-accelerated AI and machine learning workloads. This setup not only simplifies GPU management but also enables scalable, high-performance computing for modern data-driven applications.

About Us:

BI3 has been recognized for being one of the fastest-growing companies in Australia. Our team has delivered substantial and complex projects for some of the largest organizations around the globe, and we're quickly building a well-known brand for superior delivery.

> *Website: https://bi3technologies.com/*

Follow us on,
LinkedIn: https://www.linkedin.com/company/bi3technologies
Instagram: https://www.instagram.com/bi3technologies/
Twitter: https://twitter.com/Bi3Technologies

Kubernetes | Gpu Acceleration | Nvidia | Ai Infrastructure | Containers

Follow

Published in BI3 Technologies

60 followers · Last published Jan 8, 2026

Bi3 has been recognized for being one of the fastest-growing companies in Australia. Our team has delivered substantial and complex projects for some of the largest organizations around the globe and we're quickly building a brand that is well known for superior delivery.

Follow

Written by Ashwin Durairaj

11 followers · 10 following

A software developer with a passion for learning and a drive fueled by curiosity!

More from Ashwin Durairaj and BI3 Technologies

In BI3 Technologies by Ashwin Durairaj · Apr 14, 2025

Maximize GPU Utilization on Kubernetes with NVIDIA KAI Scheduler

Introduction

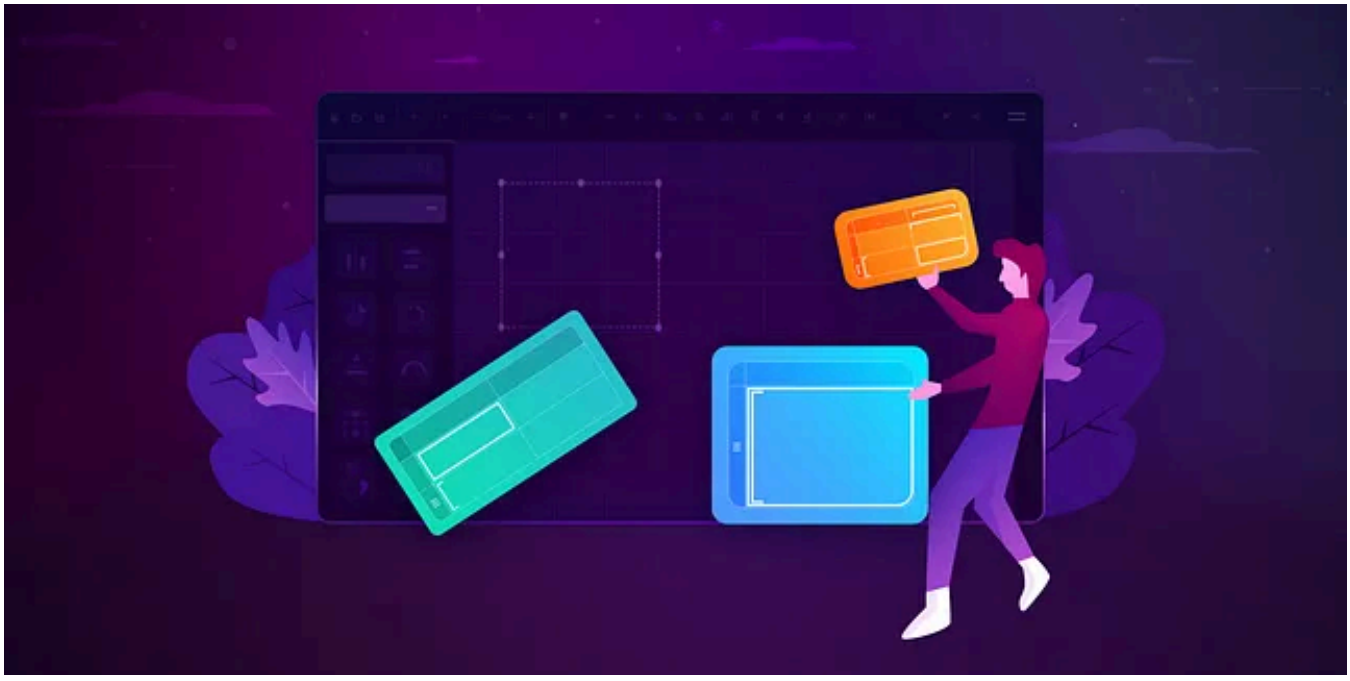
80 2

In BI3 Technologies by Naveen Kandasamy · Oct 14, 2024

Automate Power BI Dataset Refresh with Azure Data Factory

Introduction:

104 3



In BI3 Technologies by Janani Govindasamy · Sep 8, 2021

Repeating Tablix row group value on each row in Power BI report builder

When working with report builder in groupings, we frequently encounter problems with the Tablix row group value being repeated on each row...

29

In BI3 Technologies by Ashwin Durairaj · Feb 24, 2025

Retrieving Client IP in ASP.NET Core Behind Nginx Reverse Proxy

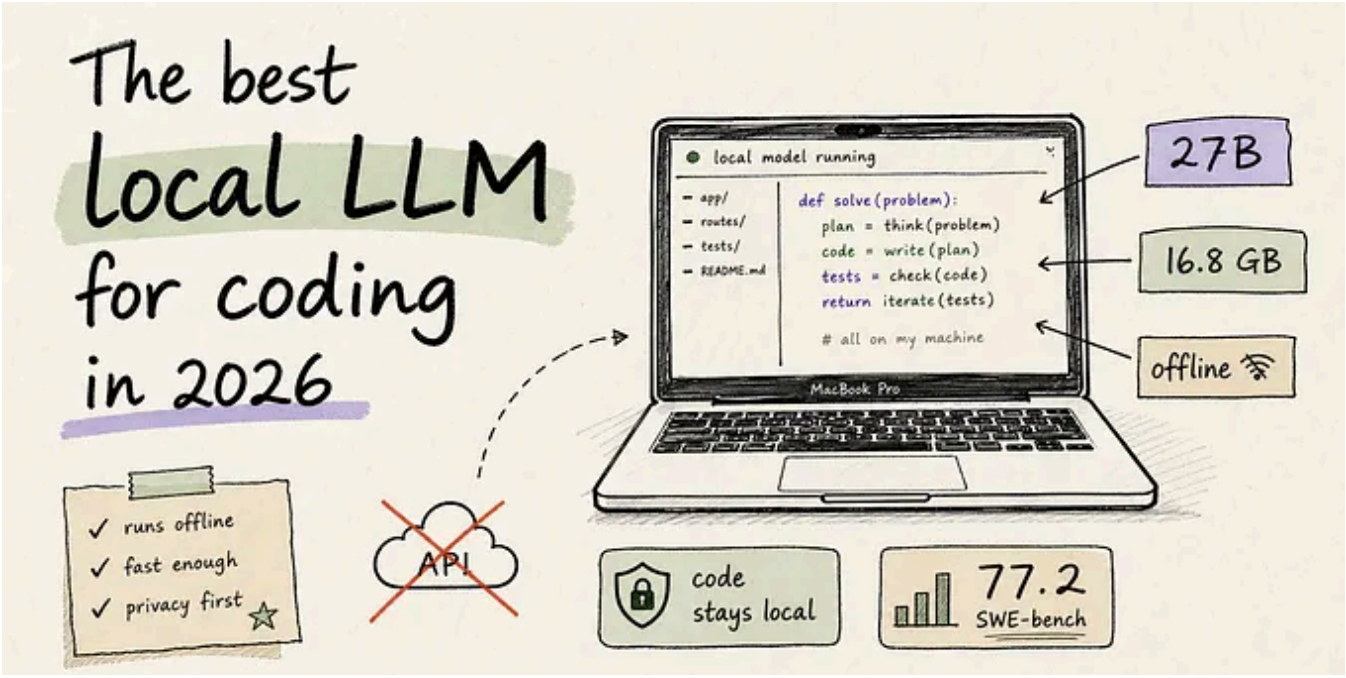
Introduction

12

See all from Ashwin Durairaj

See all from BI3 Technologies

Recommended from Medium

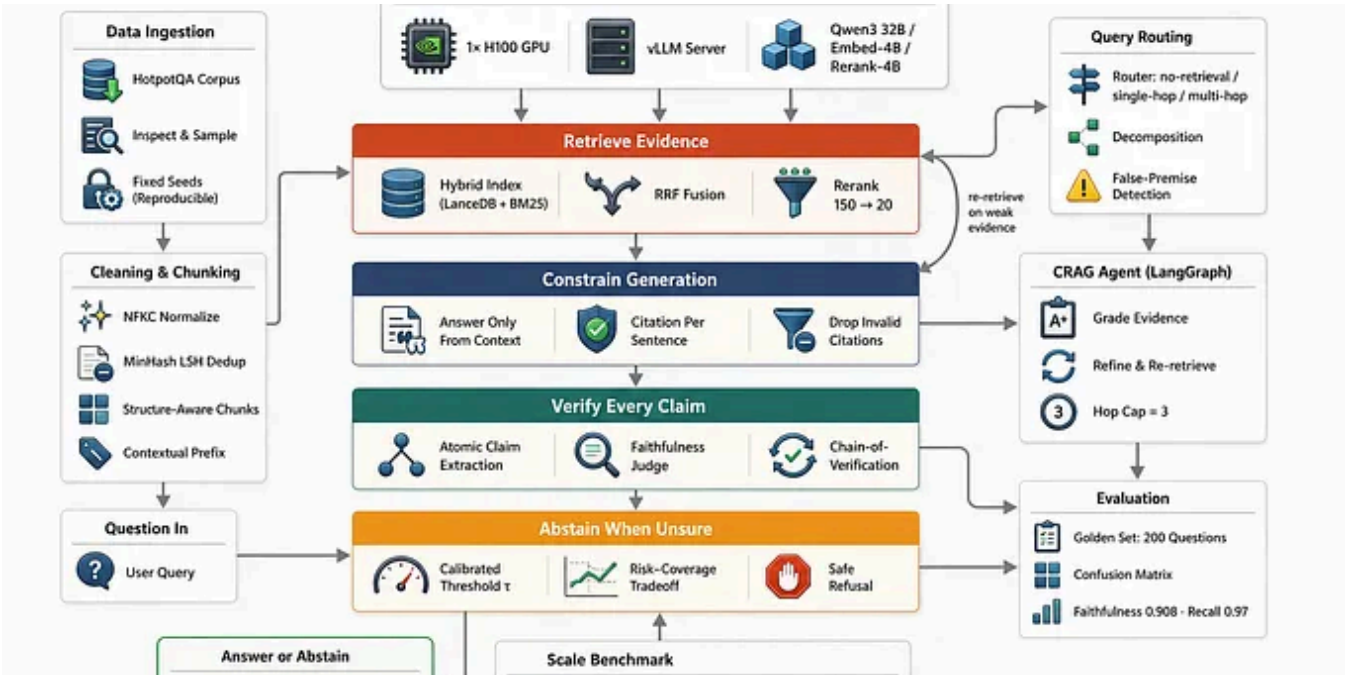


In AI Advances by Andrus · Jun 12

I Went Looking for the Best Local LLM for Coding in 2026.

In February 2025 I wrote a guide on running DeepSeek R1 locally, then quietly stopped using it within two weeks. Qwen 3.6 made me try...

882 22 10



In Level Up Coding by Fareed Khan · Jun 15

Building a RAG Pipeline for 10M+ Documents With Near-Zero Hallucination

Retrieve, constrain, verify, abstain

Agent Harnesses — Intuitively and Exhaustively Explained

A new standard for modern AI agent development

Kubernetes is Officially Doomed (And Linus Torvalds Warned Us)

Why tech giants are quietly abandoning the orchestration king, and the $10 million complexity tax your company is paying right now.

3K 112 45

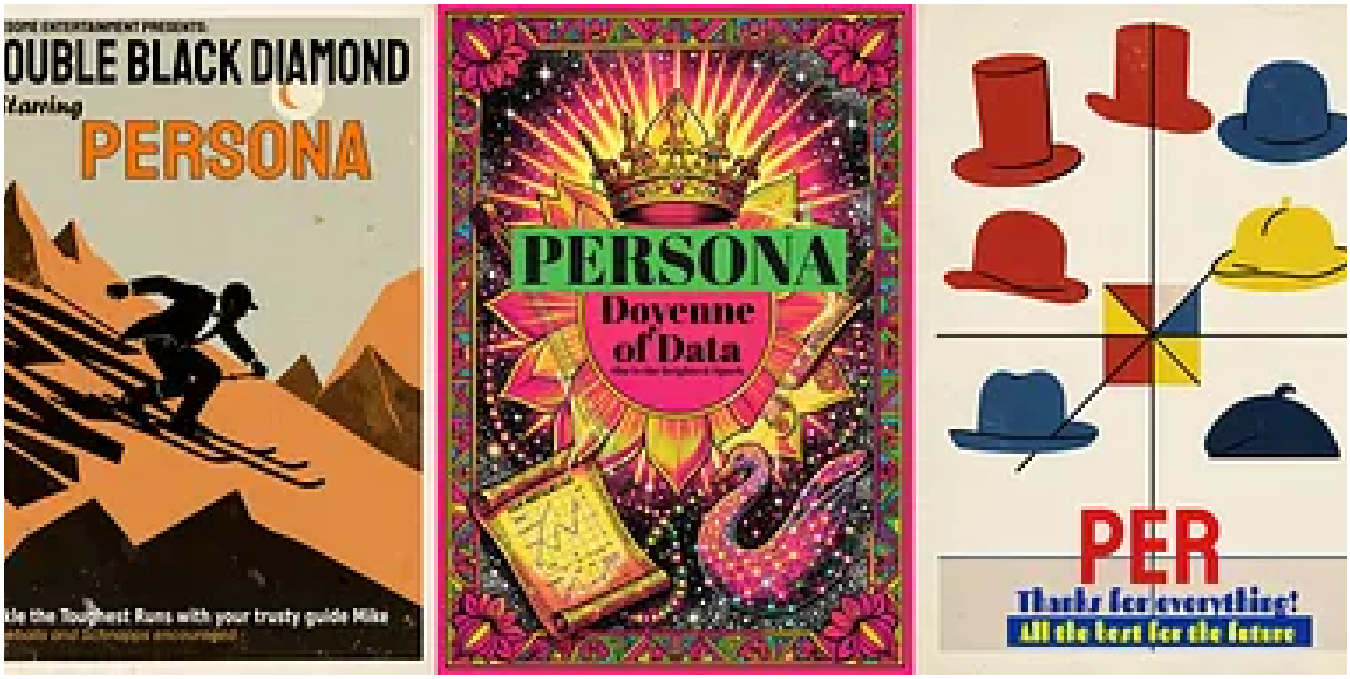
Component	Precision	Approximate Size
MoE routed experts (384 per layer x 60 layers)	INT4	~400GB
Attention / MLA (61 layers)	BF16	~120GB
Shared experts (60 layers)	BF16	~10GB
Dense MLP (layer 0)	BF16	~5GB
Embeddings + lm_head	BF16	~14GB
Total weight memory		**~549GB**

Shivank Chaudhary · Feb 19

Deploying Kimi K2.5 on H200 GPUs: The Real Story Nobody Tells You

I spent the last few days deploying Moonshot AI's Kimi K2.5 — a 1 trillion parameter Mixture-of-Experts model — on NVIDIA H200 GPUs using...

18 2



In Sparktastic by Amegilla · Apr 24

A DGX Spark Poster Generator

Many lessons learned for making best use of my DGX Spark

8

See more recommendations